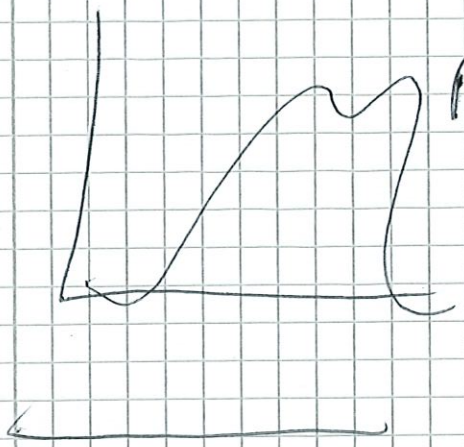


$p_{data} \approx p_{model}$

↓ NF = NLL Loss

Diffusion model -> Regression

Likelihood ratio trick

Weighted data

$x_i \in X_{data}$ $\forall\ i = 1 \dots N$

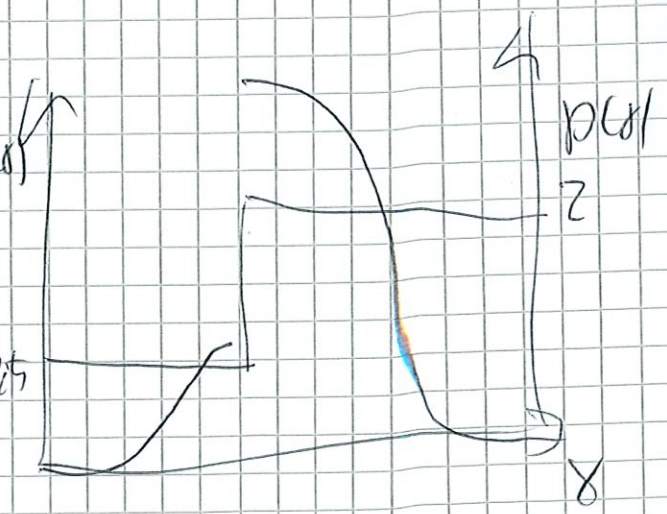
$w_i \in \mathbb{R}$ $w_i \geq 0$



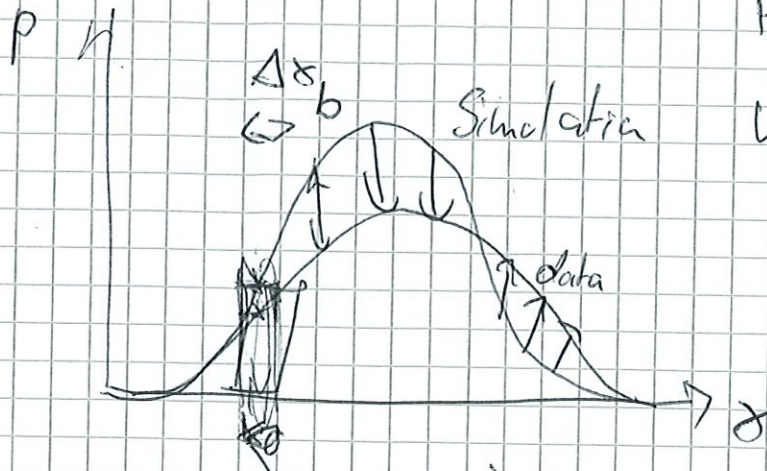
$w_i = 1$

$\rightarrow w_{1:N}$

$\sum_{i=0}^{N} w_i = N$



1



For Bin b:

$$w^{(b)} = \sum_{x_i \in b} w_i$$

$$w_b = \frac{N_{b\,data}}{N_{b\,sim}}$$

$$N_{b\,data} = \sum_{x_i \in b} 1$$

$$p(x|b) \cdot \Delta x_b$$

$$\Rightarrow \text{continuous} \quad w_b = \frac{p(x_b)_{sim} \cdot \Delta x_b}{p(x_b)_{data} \cdot \Delta x_b}$$

$$\Rightarrow \text{continuous} \quad \Delta x_b \to dx$$

$$w_b(x) = \frac{p(x)_{sim}}{p(x)_{data}} \frac{dx}{dx}$$

$$\frac{C(x)}{1-C(x)} = \frac{p(x|S)}{p(x|B)}$$

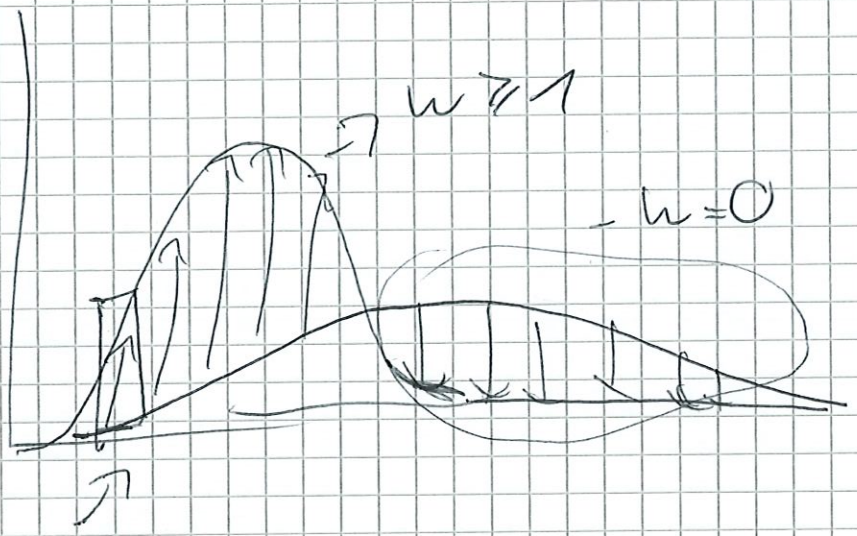
Classifier: sim $\Leftrightarrow$ data

$$\frac{S(x)}{1-C(x)} = \frac{p(x|sim)}{p(x|data)} = w(x)$$

(2)

Make 2 distributions match
- Take samples from each
- Train classifier to separate them
- Reweight one to match other, using classifier

Practical Points



one very large weight

$n_{eff} \to 1$

$\sigma_b = \sqrt{n_b}$ → Poisson

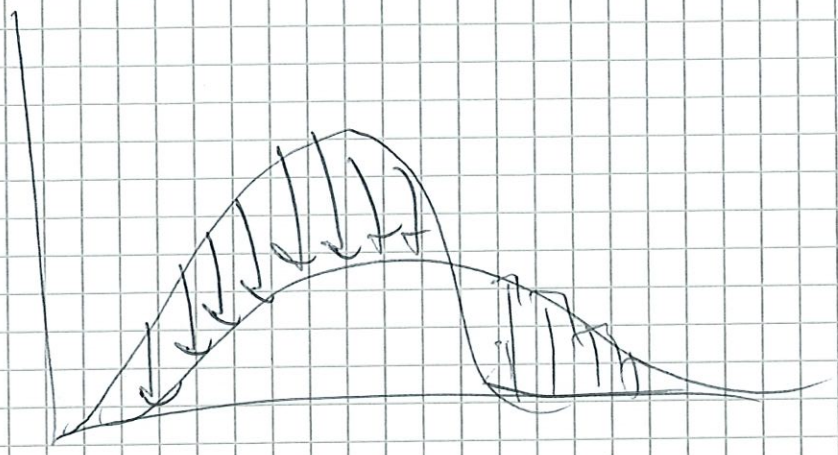
$\sigma_b = \sqrt{\sum_{i \in b} w_i^2}$ $\quad n_b = \sum_{i \in b} w_i$

$$\frac{\sqrt{n_{eff}}}{n_{eff}} = \left.\frac{\sigma_b}{n_b}\right|_{w=1} = \frac{\sqrt{n}}{n} = \frac{1}{\sqrt{n}}$$

$$\left.\frac{\sigma_b}{n_b}\right|_{w \neq 0} = \frac{\sqrt{\sum_{i \in b} w_i^2}}{\sum_{i \in b} w_i} = \frac{1}{\sqrt{n_{eff}}}$$

$\frac{1}{\sqrt{n_{eff}}}$ $\quad \forall b$

$\Rightarrow n_{eff} = \frac{(\sum_{i \in b} w_i)^2}{\sum w_i^2}$ (3)

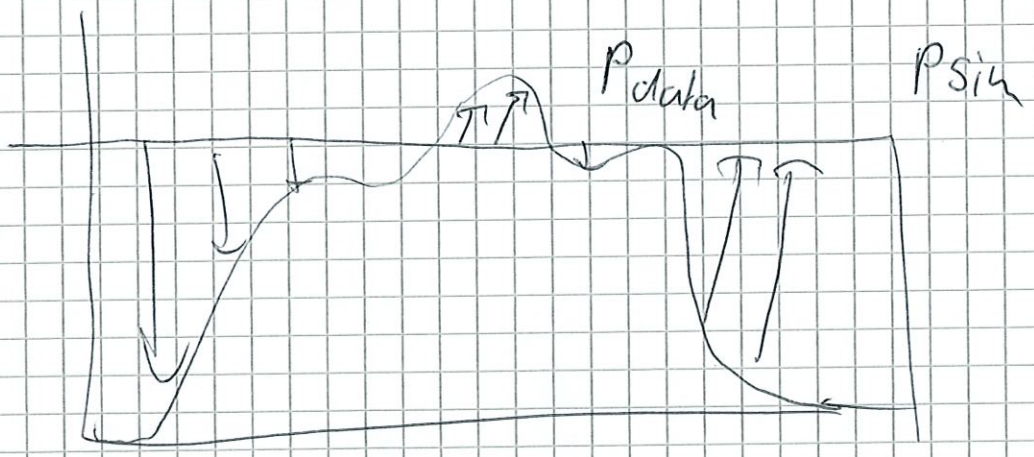


⇒ needs shade support

both: Large overlap

„Generativ"

Learn p_{sim} ⇒ can sample $x_{sim} \sim p_{sim}$



$x_{i\,sim} \sim p_{sim}$ (x) $x_j \sim p_{data}$

⇒ Train classifier

⇒ predict $x_{i\,sim} \rightarrow p_{data}$

⇒ sample new points

Transformers

→ Began in 1984 Transformer Generation
(Hasbro, Takara Tomy, ef. all)

Backbone of modern ML
→ handling sequences

Previously: vectors $\vec{x} = \begin{pmatrix} x_0 \\ \vdots \\ x_n \end{pmatrix}$

Now: Sequence of tokens, $x_1 \dots x_C$

Token
⇒ generic term for „Object a transformer uses"

$x_1 \rightarrow \vec{x}_1 = \begin{pmatrix} x_{1,0} \\ \vdots \\ x_{1,n} \end{pmatrix}$

⇒ $x_1 \in \mathbb{N}$, map $x_1 \rightarrow$ words

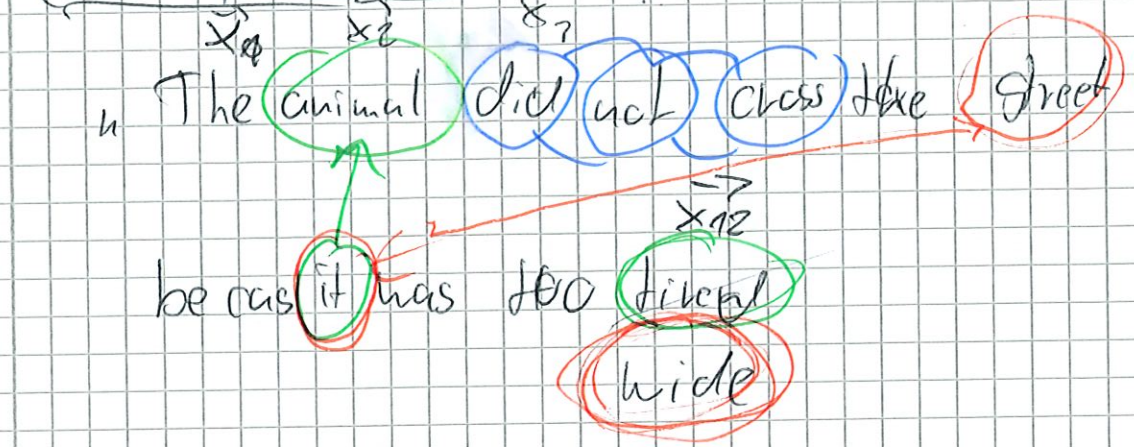
Embed Tokens

$x_i \rightarrow \mathbb{N}^n \rightarrow \vec{x}_i'$

Embedding learnable
dimension Hyperparameter

Sequence of inputs → Sequence of vectors $\vec{x}_1' \dots \vec{x}_C'$

(Self) Attention



Position is not enough -> what belongs together?

Vector Similarity? -> Dot product

$$a_{ij} = \vec{x}_i \cdot \vec{x}_j$$

$$\vec{z}_i = \sum_j a_{ij} \vec{x}_j =$$

$i=2$

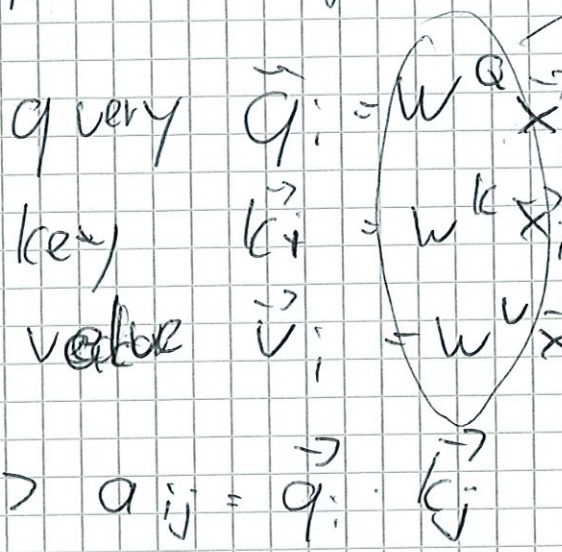
100% animal
80% it
75% tired
15% was
12% too
⋮
0% street

$\vec{x}_1$ ⋮ $\vec{x}_{12}$ → $\vec{z}_2$

6

Is "animal" really similar to "fit" in vector space? Does not have to be.

→ don't look at similarity of tokens, process them first



$\Rightarrow a_{ij} = \vec{q}_i \cdot \vec{k}_j$

⇒ How much do the keys built from my token vectors agree with the queries built from the same?

Σ More issues:

Vectors can agree by accident

→ Control variance

$$\mathrm{Var}(\vec{q}\cdot\vec{k}) = \mathrm{Var}\left(\sum_{i=1}^{d} q_i k_i\right) = \sum_{i=1}^{d} \mathrm{Var}(q_i k_i)$$

$$= \sum_{i=1}^{d} \langle q_i^2 k_i^2 \rangle - \langle q_i k_i \rangle^2$$

$$= \sum_{i=1}^{d} \langle q_i^2 \rangle \langle k_i^2 \rangle - \langle q_i \rangle^2 \langle k_i \rangle^2$$

⇒ Only care about noise for normalized inputs

$\mathrm{Var}(q_i) = \mathrm{Var}(k_i) = 1$ and $\langle q_i \rangle = \langle k_i \rangle = 0 \Rightarrow \langle q_i^2 \rangle = \langle k_i^2 \rangle = 1$

$$\Rightarrow \sum_{i=1}^{d} 1 \cdot 1 = d$$

width $\sqrt{d}$ $\Rightarrow$ scale by $\sqrt{d}$

2.) Attention across 1 row should sum to 1 and be positive

$$\Rightarrow \text{Softmax}_j(S_{ij}) = \frac{e^{S_{ij}}}{\sum_j e^{S_{ij}}}$$

$\Rightarrow$ normalize in exponential space

originally sigmoid equivalent for multi classes (all class probabilities need to add to 1)

$$a_{ij} = \text{Softmax}_j\left(\frac{\vec{q}_i \vec{k}_i}{\sqrt{d}}\right)$$

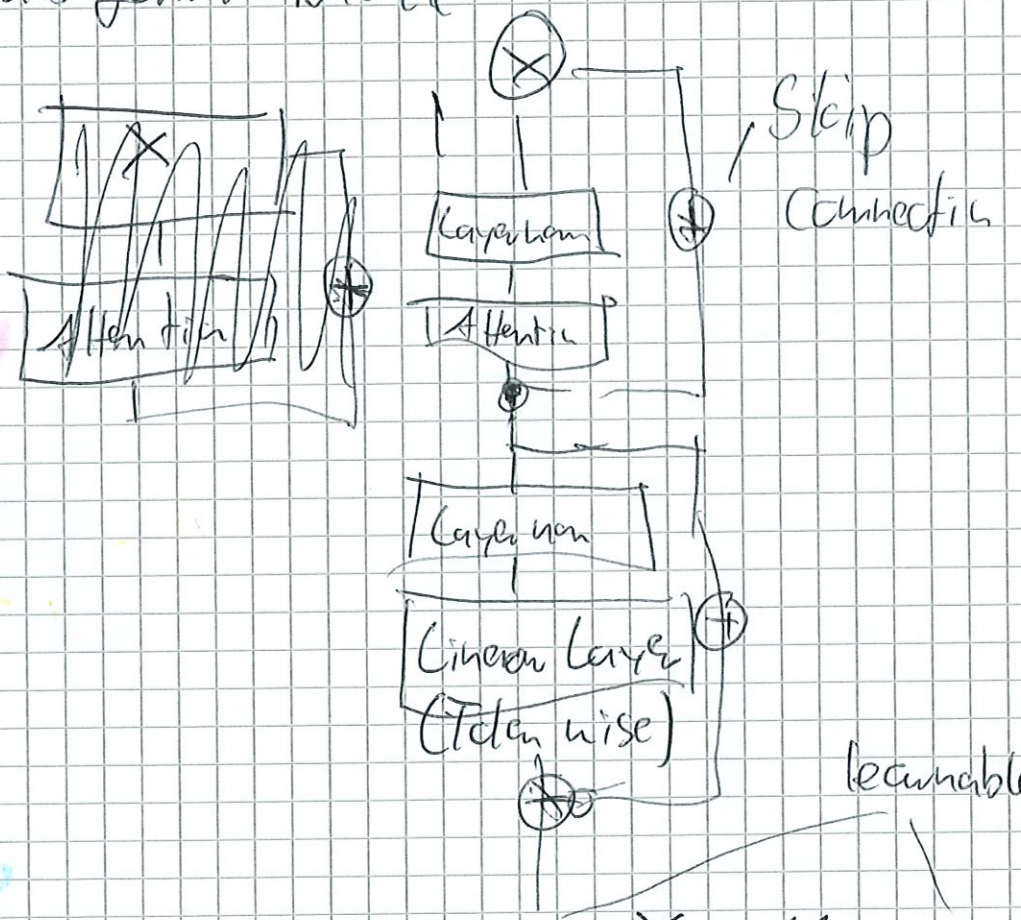
with $a_{ij} \in [0, 1]$, $\sum_j a_{ij} = 1$.

$$\vec{z}_i = \sum_{j=1}^{C} a_{ij} \vec{v}_j = \sum_j \text{Softmax}_j\left[\frac{(w^q x_i)(w^k x_j)}{\sqrt{d}}\right](w^v x_j)$$

multi-head attention

multiple Attention setups with same data input → learn bias in the end

Transformer block



learnable

$$\text{Layer norm } LN(x)_i = \gamma_i \cdot \frac{x_i - \mu}{\sigma} + \beta_i$$

$$\mu = \frac{1}{d}\sum_{i=1}^{d} x_i \quad , \quad \sigma^2 = \frac{1}{d}\sum_{i=1}^{d}(x_i - \mu)^2$$